

Can Retrieval-Augmented Few-Shot Prompting Match LoRA Fine-Tuning for Issue Report Classification? An Empirical Study

Anonymous Author(s)
Anonymous Affiliation(s)

Abstract—Classifying issue reports helps with software maintenance and triage. Fine-tuned large language models (LLMs) give the best reported results, but learning from new labels or adopting a newer model requires retraining. Retrieval-augmented few-shot prompting (RAG) needs no training but, to our knowledge, has not been systematically evaluated for this task or compared with Low-Rank Adaptation (LoRA) fine-tuning. To fill this gap, we conduct an empirical study with four Qwen2.5-Instruct models and 6,600 issue reports from eleven projects. We first test whether a query issue’s nearest neighbors predict its label by taking a similarity-weighted vote over their labels (k NN voting), which reaches 59.5% macro F_1 . RAG therefore gives these neighbors to the LLM as few-shot examples. After observing that RAG most often mislabels questions as bugs, we propose filtered RAG, which drops bug-labeled examples for suspected questions. With the same 300 labeled issues per project, RAG and filtered RAG at their best number of examples outperform LoRA fine-tuning of the same models by 2.1–6.0 points in macro F_1 at every size. When fine-tuning instead uses all eleven projects’ labeled issues, it leads RAG by 2.8 points on average and filtered RAG by 1.0 point, a difference that is not significant. With k NN voting replacing invalid LLM outputs, filtered RAG matches fine-tuning (0.1 point ahead on average). Needing only the target project’s labeled issues and 33% less peak GPU memory on average, retrieval-augmented few-shot prompting is a competitive, training-free alternative to LoRA fine-tuning.

Index Terms—Issue classification, Retrieval-augmented generation, Fine-tuning, Large language models, Mining software repositories

I. INTRODUCTION

Users and contributors of software projects report bugs, request features, and ask questions through issue trackers such as GitHub Issues [1]. Labeling each issue report with its type speeds up its resolution [2] and supports issue prioritization [3], issue assignment [4], and developer onboarding [5], [6]. Yet labels are rarely used [7] and often wrong [8], [9], and manually assigning them is laborious [10]. To address these problems, many studies propose methods for automated issue report classification (IRC) and typically evaluate them on the labels *bug*, *feature*, and *question* [7], [11]–[14].

Most IRC methods learn from labeled issues. Early work trained classic machine-learning classifiers [7], [8], [15], later work fine-tuned pre-trained transformer models [11], [16]–[18], and recent work applies large language models (LLMs) [6], [13], [14]. Among LLM-based methods, fine-tuning on labeled issues gives the best reported results [6], [14], [19], with Low-Rank Adaptation (LoRA [20]) for open models. Fine-tuning,

however, adds a training run that must be repeated to learn from newly labeled issues or to adopt a newer model [7], [10].

LLMs can also use labeled issues without training. Through in-context learning, a model infers a task from a few labeled examples in its prompt [21], and choosing the labeled examples most similar to the query issue can outperform choosing them at random [22], [23]. With such retrieval-augmented few-shot prompting (RAG), a newly labeled issue only needs to be indexed, and the LLM can be replaced without retraining. For future work, Heo and Lee [14] suggest few-shot prompting, and Aracena et al. [6] suggest retrieval augmentation to reduce the need for extensive fine-tuning and to help with ambiguous labels such as *question*. To our knowledge, however, retrieval-augmented few-shot prompting has not been systematically evaluated for IRC or compared with LoRA fine-tuning.

This paper does both. In an empirical study, we use four Qwen2.5-Instruct models of 3B to 32B parameters [24] and Heo and Lee’s balanced benchmark of issue reports from eleven projects [14], each with 300 labeled and 300 test issues. Following Heo and Lee, the retrieval index and the fine-tuning data come either from the target project’s 300 labeled issues alone (the project-specific setting) or from all 3,300 labeled issues of the eleven projects (the project-agnostic setting).

The retrieved examples are the labeled issues most similar to the query, so we first test how well their labels alone predict the query’s label. We label the query by a similarity-weighted vote over these neighbors, without an LLM (k NN voting). Using only the target project’s labeled issues, the vote reaches 59.5% macro F_1 , so the neighbors carry label information that an LLM could use. RAG therefore shows these neighbors to the LLM as examples. As with prior IRC methods [13], [25], RAG’s most frequent error is labeling questions as bugs, and we target this error by removing the bug-labeled examples for suspected questions (filtered RAG).

In this study, we answer four research questions.

RQ1: *How effectively can retrieval alone classify issue reports, and at what point does adding more retrieved neighbors stop helping?* The k NN voting baseline peaks at 59.5% macro F_1 with 15 neighbors from the target project and at 60.4% with 16 neighbors from all eleven projects. It is weakest on questions and labels 32% of them as bugs.

RQ2: *How well does retrieval-augmented few-shot prompting classify issue reports, and how does performance vary*

with the number of in-context examples? RAG shows the LLM k examples, for seven values of k from 0 (zero-shot prompting) up to 15, the bound set by k NN voting’s peaks. Every configuration with at least one example outperforms both zero-shot prompting and k NN voting’s best result. At each model’s best k , RAG reaches 69.7–76.7% macro F_1 with the target project’s labeled issues. Retrieved examples lower the share of questions predicted as bugs from 46–59% at zero-shot to 26–36% at the best k , but question remains the weakest class.

RQ3: *Can a balanced retrieval intervention reduce question-to-bug misclassification and improve retrieval-augmented few-shot performance?* The models label many questions as bugs even without examples, so we hypothesize that bug-labeled neighbors retrieved for a question reinforce this error. Filtered RAG therefore removes the bug-labeled examples from the prompt, unless they outnumber the question-labeled ones by more than a small margin. It scores higher than RAG at every $k \geq 6$ for all four models, and by 1.5–2.4 points at each method’s best k (95% confidence intervals exclude zero). It also lowers the share of questions predicted as bugs to 19–27%, at the cost of bug recall.

RQ4: *How do the two RAG variants compare with LoRA fine-tuning in classification performance, peak GPU memory, and labeled-data needs?* With the same 300 labeled issues of the target project, RAG and filtered RAG at their best k score higher than LoRA fine-tuning at every model size, by 2.1–6.0 points in macro F_1 . When fine-tuning instead uses all 3,300 labeled issues of the eleven projects, RAG trails it by 2.8 points on average over the four model sizes, and filtered RAG trails it by 1.0 point, a difference that is not statistically significant. Once a k NN voting fallback replaces invalid LLM outputs, filtered RAG matches fine-tuning (0.1 point ahead on average). Beyond macro F_1 , fine-tuning finds more of the actual bugs but over-predicts the bug label, whereas filtered RAG makes fewer false bug predictions at most model sizes. The question-to-bug confusion persists under every method we test, including fine-tuning. Both RAG variants need no training, 300 instead of 3,300 labeled issues, and 33% less peak GPU memory than fine-tuning on average.

Overall, retrieval-augmented few-shot prompting is competitive with LoRA fine-tuning while requiring no training, and the two approaches trade different strengths. Which one suits a project depends on its labeled data, its GPU budget, and whether a missed bug or a false bug label costs it more. Our main contributions are the following:

- A similarity-weighted k -nearest-neighbor baseline (k NN voting) that measures how much label information the retrieved neighbors carry, bounds the number of in-context examples, and serves as a fallback for invalid LLM outputs.
- An evaluation of retrieval-augmented few-shot prompting (RAG) for IRC across four model sizes and seven values of k .
- A training-free retrieval intervention (filtered RAG) that

reduces the question-to-bug misclassification known from prior IRC work, at the cost of bug recall.

- A comparison of RAG and filtered RAG with LoRA fine-tuning of the same models on the same labeled issues, in classification performance, peak GPU memory, and labeled-data needs, reported with paired bootstrap confidence intervals and per-class precision and recall.

The rest of the paper is organized as follows. Section II describes our methods and the fine-tuning baseline, Section III the experimental setup, and Section IV the results. Section V analyzes failure cases and practical implications, Section VI reviews related work, Section VII discusses threats to validity, and Section VIII concludes and outlines future work.

II. APPROACH

In this section, we describe the implementation details of the k NN voting baseline and our two retrieval-augmented methods (RAG and filtered RAG) as well as steps for faithfully reproducing the established LoRA fine-tuning baseline [6], [14]. Fig. 1 illustrates all four approaches to support the explanations. The remainder of the section is organized as follows: Section II-A explains the overall IRC objective. Section II-B details the indexing and retrieval pipeline shared by k NN voting, RAG, and filtered RAG. Section II-C presents k NN voting, a non-parametric baseline that classifies the query directly from its nearest neighbors. Section II-D describes RAG, our few-shot prompting technique that presents the nearest neighbors as classified examples in the prompt. Section II-E elaborates on filtered RAG’s balanced retrieval intervention applied to RAG. Finally, Section II-F details the faithful reproduction of the fine-tuning baseline.

A. Problem Formulation

Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or *question*. Only the issue’s title and description text are used for classification with no other signals from the issue tracker. The defined label space is adopted directly from prior IRC studies [6]–[8], [11], [19].

B. Indexing and Retrieval

To prepare the issues for retrieval based on textual similarity, we first embed their textual content (i.e., *title* + *description*). We do not embed the issue labels. Instead, we retain them as metadata associated with each embedded issue. To generate the embeddings, we use the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [26]. We selected this model due to its lightweight architecture, fast performance, and effectiveness on issue report text [27]. The embeddings are then indexed using the FAISS vector database library [28], which provides efficient high-dimensional similarity retrieval. At the retrieval stage, the query issue is first embedded with the same model. Next, FAISS calculates the cosine similarity of indexed issue reports to the query, and returns the top- k nearest neighbors, ordered by descending similarity.

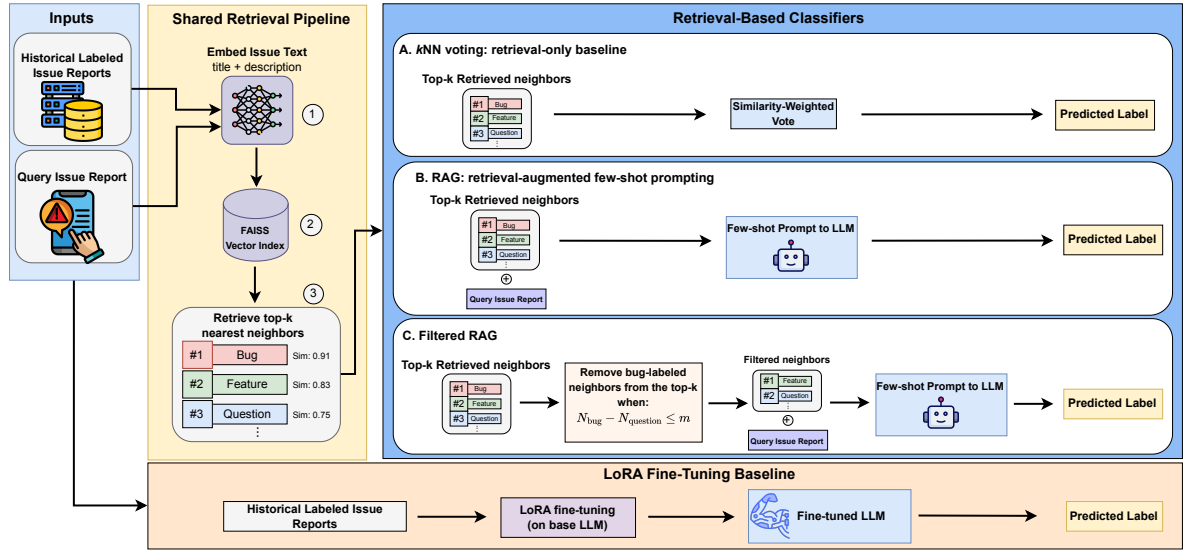


Fig. 1. Overview of the k NN voting baseline, retrieval-augmented few-shot prompting (RAG), filtered RAG, and the LoRA fine-tuning baseline.

C. k NN Voting Baseline

The k NN voting baseline labels the query by a similarity-weighted vote over the labels of its top- k neighbors, following the distance-weighted k -nearest-neighbor rule [29], [30].¹ Because it uses no LLM, k NN voting measures how much label information the neighbors carry, and its best k bounds the number of examples we show the LLM (Section IV-A). It also labels the issues whose LLM output is invalid when we evaluate complete pipelines (Section IV-D). Formally, for a query issue q with retrieved neighbors $\{(n_i, l_i, s_i)\}_{i=1}^k$, where n_i is the i -th nearest neighbor with label l_i and cosine similarity s_i to q , the predicted label $\hat{y}$ is given by:

$$\hat{y} = \arg \max_{c \in \mathcal{L}} \sum_{i=1}^k s_i \cdot \mathbf{1}[l_i = c], \quad (1)$$

where $\mathcal{L}$ is the label set, and $\mathbf{1}[l_i = c]$ is the indicator function that equals 1 when neighbor i has label c and 0 otherwise. For each label, k NN voting sums the cosine similarities of the neighbors that carry that label, and the $\arg \max$ operator returns the label with the largest total sum. It breaks ties by returning the label of the most similar neighbor in the tied classes.

D. Retrieval-Augmented Few-Shot Prompting (RAG)

RAG shows the query's top- k neighbors (Section II-B) to the LLM as labeled examples. We use the neighbors, rather than randomly chosen labeled issues, because similar examples can outperform random ones [22], [23] and because a vote over the neighbors' labels already reaches 59.5% macro F_1 (k NN voting, Section IV-A). We create the few-shot prompt in three parts as shown in Fig. 2. First, the system message frames the classification task and the label space. Second, the

¹Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

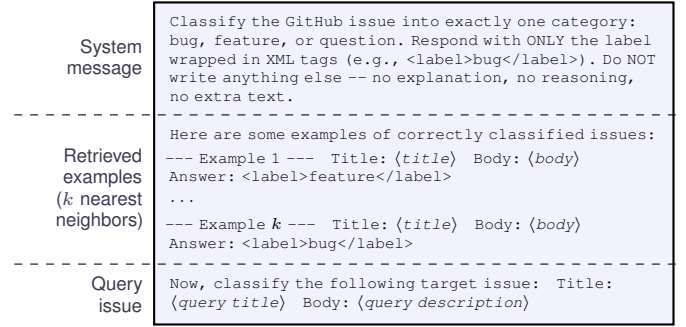


Fig. 2. The retrieval-augmented few-shot prompt structure used for RAG and filtered RAG.

top- k nearest neighbors are listed in retrieval order (descending similarity), each formatted as *title + description + label*. Finally, the query issue closes the prompt, since LLMs use information at the end of a long context better than in its middle [31]. With $k=0$, the prompt has no examples and RAG becomes zero-shot prompting, as in prior work [13].

We evaluate $k \in \{0, 1, 3, 6, 9, 12, 15\}$, up to k NN voting's peaks at $k=15$ –16 (Section IV-A). The maximum prompt sequence length is set to 8,192 tokens.² Roughly 30% of the context quota is reserved for the query issue and 70% for the neighbors. If the neighbors exceed their overall quota, they are truncated proportionally based on their length (longer neighbors get more truncation). The query issue gets truncated if it exceeds its budget as well. Truncation happens on the right side of the description text, preserving all titles and labels (if any). The LLM is instructed to generate the predicted label wrapped in XML tags (e.g. `<label>bug</label>`). The labels of the presented examples also follow the same format.

²Maximum prompt sequence length is selected empirically. 8,192 tokens offered the best trade-off between macro F_1 and inference speed across our preliminary experiments.

We use a regular expression to extract the predicted label. Outputs that contain none of the three labels are marked as *invalid*. Finally, models are loaded with Unsloth³ for memory efficiency, and the temperature is set at 0.1 for deterministic classifications.

E. Filtered RAG

Our empirical evaluations in Section IV-B reveal that RAG has the lowest class-specific F_1 on the question label, and it frequently misclassifies true-question queries as bugs, a known IRC weakness [13], [25]. Furthermore, LLM zero-shot results also show that the models are already biased towards predicting the bug label for true-question queries even before the examples are presented. Based on these insights, we hypothesize that presenting bug-labeled examples for true-question queries likely reinforces the question-to-bug misclassification trend. To address this, we propose a more balanced retrieval intervention (filtered RAG) that removes the bug-labeled neighbors whenever the query is a suspected question based on its nearest neighbors’ label distribution. Section IV-B shows that true-question queries retrieve almost balanced bug and question neighbors. Due to this balanced distribution, filtered RAG removes the bug-labeled examples from the prompt whenever $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count of neighbors with that label in the top- k and m is a small positive margin. The margin is empirically calibrated on a holdout validation set. The empirical motivations for filtered RAG and its margin calibration are fully detailed in Section IV-B. Other than the balanced retrieval intervention, filtered RAG inherits RAG’s full setup and configurations.

F. LoRA Fine-Tuning Baseline

Recent IRC approaches have achieved strong performance by supervised fine-tuning of LLMs [6], [14], [19], using Low-Rank Adaptation (LoRA) [20]. In these studies, the model is trained on labeled issue reports and predicts the labels for unseen test issues. Each training example is formatted as an instruction prompt that includes the issue’s title and description as input, and its label as output. We adopt the instruction-style prompt template used in these prior LoRA fine-tuning studies [6], [14] verbatim, rather than designing our own, so that the baseline faithfully reproduces their established methodology. At inference, the trained model receives the test issue in the same format, and predicts the empty label field. Regular expressions are used to extract the prediction from the output, and outputs that contain none of the three labels are marked as *invalid*. We use this established methodology as our fine-tuning baseline.

Training hyperparameters follow prior work [6], [14]: The LoRA rank and alpha are set to 16, with a learning rate of 2×10^{-4} and the paged AdamW 8-bit optimizer. Training runs for 3 epochs with a per-device batch size of 1 and gradient accumulation steps of 16. The maximum prompt sequence length is set to 2,048 tokens, since 94.9% of the test issues in the dataset will not be truncated at this length. Issues that

exceed this length get truncated from the right side of their description text to fit. Similar to RAG and filtered RAG, the models are loaded with Unsloth and the temperature is set at 0.1.

III. EXPERIMENTAL SETUP

Dataset and Settings: We use the dataset introduced in Heo et al. [14] for our experiments. The dataset has a total of 6,600 issue reports gathered from eleven active open-source projects: `ansible`, `bitcoin`, `dart-lang`, `roslyn`, `react`, `flutter`, `kubernetes`, `TypeScript`, `vscode`, `opencv`, and `tensorflow`. Each project has 600 issue reports with an equal distribution across the labels (*bug*, *feature*, and *question*). To support their LLM fine-tuning method, Heo et al. divided the dataset into 50/50 train and test splits, stratified by project and label. This results in 300 train and 300 test issue reports per project. We also adopt Heo et al.’s project-specific (PS) and project-agnostic (PA) configurations to examine how data scope affects IRC performance. In the PS setting, each project’s train and test data are used in isolation, resulting in eleven different train-test pairs. In the PA setting, the train splits of all projects are merged into a single training set of 3,300 issue reports. In both configurations, RAG and k NN voting only use the training splits as their retrieval index to ensure fair comparison with the fine-tuning baseline.

Models: We use the Qwen2.5-Instruct model family [24] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance on general-purpose benchmarks. Using the same model with different parameter sizes enables analysis of the effect of model scale in LLM-based IRC approaches.

Evaluation Metrics: We report the macro-averaged F_1 over the three classes (*bug*, *feature*, *question*) as the headline metric and use it to select each method’s best configuration. We report scores in percent and differences between methods in percentage points (points). We also report macro-averaged and per-class precision and recall. All metrics are calculated over the total 3,300-issue test set. We also report the rate of outputs that fail to parse to a valid label (*invalid rate*). Invalid outputs are counted as incorrect predictions. For statistical significance checks, we report paired bootstrap 95% confidence intervals on the macro F_1 differences between methods.

Hardware Setup: Fine-tuning of Qwen-14B and Qwen-32B and the Qwen-32B runs at $k \geq 12$ used one NVIDIA L40 or L40S GPU (48 GB VRAM). All other experiments used one NVIDIA RTX 4090 GPU (24 GB VRAM).

IV. EVALUATION

In this section, we present the results of our experiments and answer the four research questions posed in Section I, one per subsection. Throughout this section, a method’s *best* k or configuration is the one with the highest macro F_1 . Fig. 3 reports macro F_1 across the k grid for k NN voting, RAG, and filtered RAG, and Table I lists every method at its best

³<https://unsloth.ai>

configuration, including the per-class precision and recall that RQ2–RQ4 refer to.

A. RQ1: Effectiveness of Retrieval-Only IRC

We evaluate k NN voting in PS and PA at $k \in \{1, \dots, 20, 25, 30\}$ (Fig. 3a). Similarity-weighted voting achieves its highest observed macro F_1 of 59.5% at $k=15$ in PS and 60.4% at $k=16$ in PA. PA has slightly higher scores at every evaluated k . The small performance difference is consistent with the substantial overlap between retrieved neighbors: PS and PA share 84–90% of their top- k neighbors for the evaluated values between 3 and 30. Thus, even when the retrieval index includes all 3,300 training issues, most retrieved neighbors belong to the query’s project.

Question has the lowest class-specific F_1 at every evaluated k in both settings. At the peak configurations, 32% of question issues are predicted as bugs in both settings, compared with 18% predicted as features in PS and 17% in PA. Bug recall exceeds precision at every evaluated k , indicating that k NN voting over-predicts this class. At the peak configurations, bug recall is approximately 73% and precision is 55–56% (Table I).

Retrieval-only voting achieves peak macro F_1 scores of 59.5% with project-specific data and 60.4% with pooled data. The small difference accompanies substantial overlap between retrieved neighbors. Question is the weakest class, with misclassifications more frequently assigned to bug than feature.

B. RQ2: Effectiveness of Retrieval-Augmented Few-Shot IRC

We evaluate RAG at $k \in \{0, 1, 3, 6, 9, 12, 15\}$ in PS and PA, where $k=0$ is the zero-shot baseline [13]. The peaks of k NN voting at $k=15$ –16 (RQ1) set the upper end of this grid. Fig. 3b reports macro F_1 across the grid in both settings.

PS has slightly higher macro F_1 than PA at almost every (model, k) pair with $k \geq 1$, using 300 in-project retrieval examples rather than a pool of 3,300 examples. Zero-shot predictions are identical between settings because no retrieval is performed. The small PS–PA differences are consistent with the neighbor overlap observed in RQ1. We use PS for the remaining RAG analysis.

All 48 nonzero- k configurations, spanning four model sizes and both data scopes, outperform the corresponding zero-shot baseline and the strongest k NN voting configuration in macro F_1 , macro precision, and macro recall. Thus, LLM classification using retrieved examples outperforms similarity-weighted voting. Within the PS grid, the best k is 3, 6, 12, and 12 for Qwen-3B, -7B, -14B, and -32B, respectively. These configurations achieve 69.7–76.7% macro F_1 , averaging a gain of 7.6 points over zero-shot. Increasing k from 12 to 15 does not improve any model’s PS score.

Question remains the weakest class by F_1 at every evaluated k in both settings. It has the lowest recall but never the lowest precision. At zero-shot, 46–59% of true-question issues are predicted as bugs, compared with 5–16% predicted as features. These question-to-bug errors account for roughly 80% of false

bug predictions, linking the question-recall deficit to low bug precision.

Retrieved examples reduce this confusion. At the best PS configurations, 26–36% of question issues are predicted as bugs and 9–15% as features. Averaged across the four models, the share of issues predicted as bug falls from 51% to 42%, bug precision increases from 57.5% to 66.2%, and question recall from 35.0% to 51.4% relative to zero-shot (Table I). The increase in question-class F_1 accounts for 56–70% of the macro- F_1 improvement across model sizes.

The zero-shot results show that question-to-bug confusion exists before any examples are presented. We hypothesize that bug-labeled examples retrieved for true-question queries reinforce this tendency. Based on this intuition, we propose a more balanced retrieval technique as an intervention that removes the bug-labeled neighbors whenever the query is a suspected question. We base the suspicion signal on the nearest neighbors’ label distribution and calibrate it without including the test set. From each project’s total 300 training issues, we assign 30 issues stratified by label as queries and 270 as the retrieval index, mirroring the PS setting. On this validation split, true-question queries retrieve almost balanced bug and question examples: averaged across $k \in [1, 15]$, the mean of $N_{\text{bug}} - N_{\text{question}}$ is -1.36 . We therefore treat a query as a suspected question when $N_{\text{bug}} - N_{\text{question}} \leq m$, where N_{label} is the count of neighbors with that label in the top- k and m is a small positive margin. Sweeping $m \in \{1, 2, 3, 4, 5\}$ ⁴ on the validation split and averaging across $k \in [1, 15]$, $m=3$ had a more favorable trade-off against the other values, firing for 89.1% of true-question and 56.7% of true-bug queries. We adopt $m=3$ and refer to the resulting variant as filtered RAG.

Retrieved examples improve macro F_1 , macro precision, and macro recall over zero-shot prompting and retrieval-only voting in every evaluated configuration with $k \geq 1$. The best PS configurations achieve 69.7–76.7% macro F_1 , and no model improves beyond $k=12$. Question recall remains the principal weakness: 26–36% of question issues are still classified as bugs, motivating the balanced retrieval intervention evaluated in RQ3.

C. RQ3: Effectiveness and Trade-offs of the Balanced Retrieval Method

We compare filtered RAG with RAG under PS at the same $k \in \{1, 3, 6, 9, 12, 15\}$. At every tested $k \geq 6$, filtered RAG has higher macro F_1 for all four model sizes (Fig. 3c), by at least 1.2 points. At $k=1$ and $k=3$, its scores are lower by up to 1.0 point. With the fixed margin $m=3$, the condition $N_{\text{bug}} - N_{\text{question}} \leq 3$ holds for every query at these two small k values. Bug-example removal is therefore unconditional, leaving no demonstrations for 40% of queries at $k=1$ and 18% at $k=3$.

⁴We cap the sweep at $m=5$, since it is the 95th percentile of $N_{\text{bug}} - N_{\text{question}}$ on true-question queries.

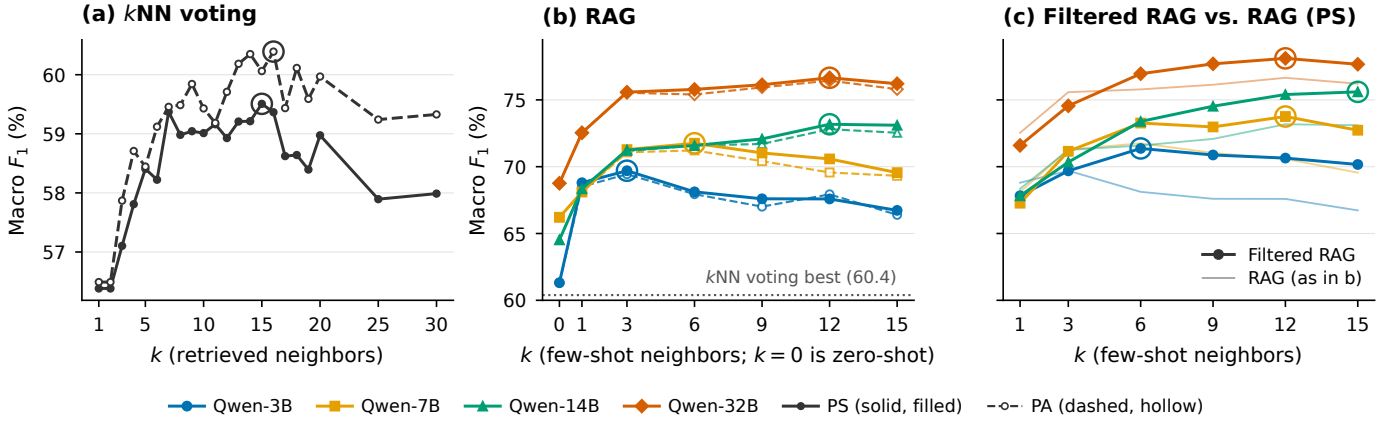


Fig. 3. Macro F_1 (%) across the k grid. (a) k NN voting in PS (solid) and PA (dashed). (b) RAG for the four Qwen sizes in PS (solid, filled) and PA (dashed, hollow). (c) Filtered RAG (markers) against RAG. (b) and (c) share one y -axis and rings mark each method’s best k .

TABLE I
POOLED TEST-SET RESULTS (%) OF EVERY METHOD AT ITS BEST CONFIGURATION.

Model	Method	k	Bug		Feature		Question		P_{mac}	R_{mac}	Macro F_1	Invalid
			P	R	P	R	P	R				
–	k NN voting (PS)	15	55.2	73.4	67.7	56.2	59.2	49.8	60.7	59.8	59.5	0.0
	k NN voting (PA)	16	55.6	73.1	68.9	57.9	60.0	50.8	61.5	60.6	60.4	0.0
Qwen-3B	Zero-shot	0	54.3	91.5	88.6	61.0	56.8	35.3	66.6	62.6	61.3	0.2
	RAG (PS)	3	63.8	80.3	79.4	81.3	69.4	49.4	70.9	70.3	69.7	0.2
	Filtered RAG (PS)	6	72.3	66.5	79.4	81.3	64.4	64.6	72.0	70.8	71.4	1.8
	Fine-tuning (PS)	–	60.6	87.0	77.0	79.2	79.3	40.0	72.3	68.7	67.6	1.1
	Fine-tuning (PA)	–	63.7	87.3	83.3	75.1	72.4	51.1	73.1	71.2	70.8	0.8
Qwen-7B	Zero-shot	0	57.5	88.1	79.2	78.9	78.3	36.7	71.7	67.9	66.2	0.1
	RAG (PS)	6	65.7	82.4	78.7	84.8	83.8	47.4	76.1	71.5	71.8	3.5
	Filtered RAG (PS)	12	71.3	77.4	76.1	85.5	82.5	55.8	76.6	72.9	73.8	3.8
	Fine-tuning (PS)	–	65.3	63.5	75.9	76.9	61.9	62.7	67.7	67.7	67.7	0.1
	Fine-tuning (PA)	–	67.2	87.8	86.8	82.5	80.0	58.8	78.0	76.4	76.2	0.3
Qwen-14B	Zero-shot	0	57.0	87.6	76.1	85.5	87.3	29.4	73.5	67.5	64.5	0.1
	RAG (PS)	12	65.1	84.4	83.9	83.6	85.5	48.9	78.2	72.3	73.2	4.5
	Filtered RAG (PS)	15	69.6	80.3	83.6	84.4	83.2	57.9	78.8	74.2	75.6	4.7
	Fine-tuning (PS)	–	70.0	74.0	82.8	69.2	60.9	66.9	71.2	70.0	70.4	0.3
	Fine-tuning (PA)	–	73.0	86.5	88.2	78.1	76.4	71.0	79.2	78.5	78.5	0.0
Qwen-32B	Zero-shot	0	61.1	86.1	76.6	86.9	85.7	38.7	74.5	70.6	68.8	0.2
	RAG (PS)	12	70.1	82.8	85.5	82.8	84.0	59.8	79.9	75.2	76.7	4.6
	Filtered RAG (PS)	12	74.8	80.4	84.7	83.3	80.8	66.4	80.1	76.7	78.1	4.0
	Fine-tuning (PS)	–	74.4	70.0	81.4	82.5	66.5	69.4	74.1	74.0	74.0	0.1
	Fine-tuning (PA)	–	68.9	92.9	83.2	87.6	89.5	53.4	80.5	78.0	77.1	0.1

PS: 300 labeled issues of the target project; PA: the 3,300 issues of all eleven projects. P/R : precision/recall; $P_{\text{mac}}/R_{\text{mac}}$: their macro averages. Invalid outputs count as incorrect. Bold: best macro F_1 per model size.

Comparing each method’s best configuration, filtered RAG increases macro F_1 by 1.5–2.4 points (Table I). The 95% CIs of these gains exclude zero for all four models: $[+0.2, +3.0]$, $[+0.7, +3.2]$, $[+1.4, +3.4]$, and $[+0.7, +2.2]$ points⁵. Filtered RAG’s selected k increases for the three smaller models and remains 12 for Qwen-32B. When the trigger fires, filtered RAG removes all bug examples from the top- k , so a higher initial k is needed to leave enough context for the LLM.

The class-level results expose the trade-off. Question F_1 increases by 3.0–6.8 points: recall rises by 6.5–15.3 points while precision falls by 1.2–5.0 points. Bug precision increases

by 4.5–8.6 points, but bug recall decreases by 2.5–13.8 points at every model size. The largest recall loss occurs for Qwen-3B, from 80.3% to 66.5%, the one model where the intervention over-corrects; its bug F_1 falls from 71.1% to 69.3%. Bug F_1 improves for the three larger models. Feature F_1 changes by at most 1.1 points. The full precision and recall values appear in Table I.

At these configurations, question-to-bug error rates decrease from 26–36% with RAG to 19–27% with filtered RAG. Question-to-feature error rates change by at most 1.4 points. Averaged across $k \in [1, 15]$, the question-to-bug rate drops from 31–40% to 24–36%. The improvement is therefore a targeted reduction of question-to-bug confusion, at the cost of

⁵1,000 paired bootstrap resamples per model.

TABLE II
MACRO- F_1 DIFFERENCE FROM PA FINE-TUNING IN PERCENTAGE POINTS, WITHOUT AND WITH THE k NN VOTING FALLBACK (PAIRED BOOTSTRAP 95% CI).

Model	RAG		Filtered RAG	
	ΔF_1	95% CI	ΔF_1	95% CI
Qwen-3B	-1.1	[-2.8, +0.7]	+0.5	[-1.2, +2.4]
+ fallback	-1.5	[-3.2, +0.2]	+0.9	[-0.8, +2.8]
Qwen-7B	-4.4	[-5.9, -2.9]	-2.4	[-3.9, -0.9]
+ fallback	-3.3	[-4.7, -1.7]	-1.1	[-2.6, +0.4]
Qwen-14B	-5.4	[-6.8, -3.9]	-2.9	[-4.4, -1.3]
+ fallback	-4.0	[-5.4, -2.5]	-1.4	[-3.0, +0.1]
Qwen-32B	-0.5	[-2.0, +0.9]	+1.0	[-0.5, +2.4]
+ fallback	+0.8	[-0.8, +2.2]	+2.1	[+0.6, +3.5]
All sizes	-2.8	[-3.9, -1.8]	-1.0	[-2.0, +0.1]
+ fallback	-2.0	[-3.0, -1.0]	+0.1	[-0.9, +1.1]

RAG and filtered RAG under PS at their best k , fine-tuning under PA; negative values favor fine-tuning. + fallback: invalid outputs of both methods labeled by k NN voting. Bold: CI excludes zero. All sizes: mean of the four per-model differences.

bug recall. Invalid outputs remain counted as incorrect; their rates at these same configurations are reported in Table I and analyzed in RQ4.

Filtered RAG outperforms RAG at every $k \geq 6$ for all four model sizes. The gain is a targeted reduction of question-to-bug misclassifications: question F_1 rises by 3.0–6.8 points and bug precision by 4.5–8.6 points, at the cost of bug recall. Bug F_1 improves for the three larger models and drops by 1.8 points for Qwen-3B; feature F_1 changes by at most 1.1 points.

D. RQ4: Few-Shot Prompting vs. LoRA Fine-Tuning for IRC

Classification Performance: We fine-tune all four models under both data scopes and compare them with RAG and filtered RAG at their best k (Table I). When all methods use the same 300 labeled issues per project (PS), both RAG variants outperform fine-tuning at every model size: RAG leads by 2.1–4.0 points in macro F_1 , and filtered RAG by 3.8–6.0 points. Fine-tuning benefits from pooling the 3,300 labeled issues across all eleven projects (PA), which changes both the number of training examples and their project composition. Pooling raises its macro F_1 by 3.1–8.4 points relative to PS, consistent with prior findings [14], although the class-level effects vary: for Qwen-32B, it increases bug recall but lowers question recall (Table I).

The larger pool does not provide a corresponding benefit for retrieval (Section IV-B). We therefore use PS for RAG and filtered RAG and PA for fine-tuning in the remaining comparisons, giving each method its better-performing data scope. The eleven PS indexes collectively hold the same 3,300 training issues as the PA training set, so the two scopes differ in the data available per target project, not in the total annotation budget.

We report macro F_1 differences as the RAG variant minus fine-tuning, so negative values favor fine-tuning (Table II). Aggregated across the four model sizes, RAG trails fine-tuning

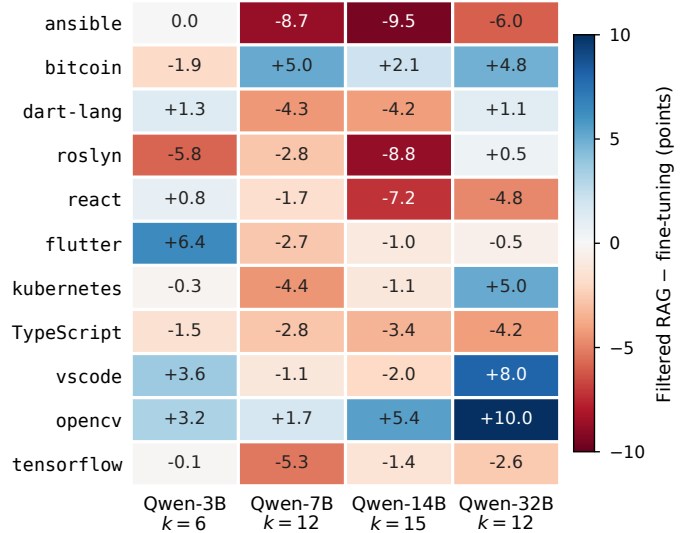


Fig. 4. Macro F_1 difference in points between filtered RAG (PS, best k) and PA LoRA fine-tuning on each project’s 300 test issues, for the four model sizes. Blue favors filtered RAG, red favors fine-tuning.

by 2.8 points (paired bootstrap 95% CI [-3.9, -1.8]) and filtered RAG by 1.0 point, whose CI [-2.0, +0.1] includes zero.⁶ RAG trails fine-tuning at every model size, whereas filtered RAG shifts the difference in its favor at every size, to between 2.9 points behind and 1.0 point ahead. Fine-tuning is significantly ahead at Qwen-7B and Qwen-14B. At Qwen-3B and Qwen-32B, the confidence intervals include zero, and filtered RAG has the higher point estimates. Results also vary by project: filtered RAG matches or exceeds PA fine-tuning in 16 of the 44 (project, model) pairs, including all four sizes on *opencv* and three of four on *bitcoin* (Fig. 4).

Fine-tuning detects more actual bugs at every model size, while filtered RAG makes fewer false bug predictions at three of the four sizes. Like the zero-shot models (Section IV-B), the PA fine-tuned models over-predict bug: they assign this label to 39–46% of test issues, although only 33% are bugs. Their bug recall is high (86.5–92.9%), but precision is lower (63.7–73.0%), and 23–36% of question issues are classified as bugs. Filtered RAG predicts bug for 31–38% of issues, closer to the true share, and misclassifies 19–27% of questions as bugs. Its bug recall is lower (66.5–80.4%), while its precision is generally higher (69.6–74.8%). The one size at which filtered RAG does not make fewer false bug predictions is Qwen-14B, where fine-tuning’s tendency to over-predict bug is less pronounced: it labels 39.5% of issues as bugs and misclassifies 23% of questions as bugs. A bug share close to the true proportion is not by itself evidence of quality; what matters is the trade-off between bug recall and bug precision, and which method is preferable depends on the relative cost of missed bugs and incorrect bug assignments in a given project.

At Qwen-7B and Qwen-14B, filtered RAG’s macro recall is lower than fine-tuning’s by 3.5 and 4.3 points, respectively,

⁶We use 1,000 paired bootstrap resamples per comparison.

TABLE III
COMPUTATIONAL COST OF k NN VOTING (PS, $k=15$), RAG, FILTERED RAG, AND LoRA FINE-TUNING AT THE CONFIGURATIONS OF TABLE II. RAM IS THE OBSERVED PEAK GPU MEMORY; TRAIN AND INFER ARE WALL-CLOCK RUNTIMES, AND TOTAL IS THEIR SUM.

Model	Method	RAM (GB)	Train (h)	Infer (h)	Total (h)
–	k NN voting	0.2	–	<0.01	<0.01
Qwen-3B	RAG	2.9	–	0.18	0.18
	Filtered RAG	2.9	–	0.22	0.22
	Fine-tuning	5.5	0.31	0.11	0.42
Qwen-7B	RAG	6.8	–	0.50	0.50
	Filtered RAG	6.8	–	0.60	0.60
	Fine-tuning	9.9	0.48	0.10	0.58
Qwen-14B	RAG	12.6	–	1.37	1.37
	Filtered RAG	12.6	–	1.35	1.35
	Fine-tuning	16.7	1.49	0.22	1.71
Qwen-32B	RAG	22.3	–	4.62	4.62
	Filtered RAG	22.3	–	4.15	4.15
	Fine-tuning	31.5	2.28	0.44	2.71

while its macro precision is within 1.4 points of fine-tuning’s at every size. Invalid outputs contribute to this recall deficit: they count as false negatives for the true class without adding a prediction to any valid class’s precision denominator. At the selected configurations, invalid rates are 0.2–4.6% for RAG, 1.8–4.7% for filtered RAG, and at most 0.8% for fine-tuning. For filtered RAG at Qwen-7B and larger sizes, 6–8% of true bugs receive invalid outputs.

We therefore compare complete pipelines that assign a valid label to every issue by applying a k NN voting fallback only when the LLM output is invalid. The fallback uses PS voting at fixed $k=15$ for RAG/filtered RAG and PA voting at fixed $k=16$ for fine-tuning. It uses the original, unfiltered neighbors, independently of the number of demonstrations shown to the LLM. The retrieval-based pipelines can reuse their cached neighbor pool; adding this fallback to fine-tuning introduces a retrieval step. No additional LLM calls are required.

After applying the fallback to all three LLM approaches (Table II, + fallback rows), filtered RAG’s differences from fine-tuning range from 1.4 points behind to 2.1 points ahead. Its deficits at Qwen-7B and Qwen-14B are approximately halved, to 1.1 and 1.4 points, while its Qwen-32B advantage increases to 2.1 points. Fine-tuning’s scores change by at most 0.3 points. The aggregate difference between filtered RAG and fine-tuning becomes 0.1 point in filtered RAG’s favor (95% CI $[-0.9, +1.1]$), and per model the interval excludes zero only at Qwen-32B, in filtered RAG’s favor.

Computation and Data Cost: Our RAG variants require 47%, 31%, 25%, and 29% less peak GPU memory than LoRA fine-tuning for Qwen-3B, -7B, -14B, and -32B, respectively (33% on average), and remain competitive with it while using $11\times$ less labeled data per project (Table III). They have no training phase, but their longer prompts make each classification slower. Fine-tuned models label the 3,300 test issues in 0.10–0.44 h, whereas filtered RAG takes 0.22–4.15 h. Including training, filtered RAG takes less total time than fine-tuning at Qwen-3B and Qwen-14B, about the same at Qwen-7B, and

more at Qwen-32B (4.15 h versus 2.71 h). Fine-tuning must, however, be repeated whenever the model is replaced or new labeled issues are added, whereas retrieval only needs to index the new issues. Without an LLM, k NN voting labels the 3,300 test issues in under 4 s with 0.2 GB of GPU memory, at a macro F_1 of 59.5% (Section IV-A).

Given the same 300 labeled issues per project, both RAG variants outperform LoRA fine-tuning at every model size (RAG by 2.1–4.0 points in macro F_1 , filtered RAG by 3.8–6.0 points). With $11\times$ more labeled data per project, fine-tuning leads RAG by 2.8 points in aggregate, but its 1.0-point lead over filtered RAG is not statistically significant. Once a k NN voting fallback handles invalid outputs, filtered RAG closes this gap (0.1 point ahead) and is significantly better at Qwen-32B. The methods trade different errors: fine-tuning detects more bugs but over-predicts the bug class, whereas filtered RAG makes fewer false bug predictions. Retrieval-augmented few-shot prompting also requires 33% less peak GPU memory than fine-tuning on average, but its inference is slower, and at Qwen-32B its total runtime is higher.

V. DISCUSSION

A. Failure Analysis

In this section, we conduct a manual qualitative analysis to better understand the misclassification patterns across the explored IRC methods. We inspect the misclassifications produced by Qwen-32B under each method’s best configuration. From the full set of misclassifications produced by RAG, filtered RAG, and LoRA fine-tuning, we randomly sampled 50 misclassified instances per method, yielding 150 instances in total. Each method’s sample includes 30 true-question, 10 true-bug, and 10 true-feature errors. The 30/10/10 split mimics the empirical class distribution of misclassifications pooled across the three methods. Each case was analyzed by two authors independently, with any discrepancies resolved through discussions. The observed failure categories and their frequencies are as follows:

Wrong template format (77.33%): Issue reports whose content and ground-truth label contradict their chosen template and structure. All of these errors are from questions and feature requests that the user filed under a bug-report template (*Steps to reproduce*, *Expected*, *Actual*, *System Information*). Consequently, the structure of the bug template misleads the LLM classifier towards predicting bug, regardless of the issue’s content. For example, `vscode#193985`⁷ is an actual support question that opens with a literal `Type: Bug` because it uses VS Code’s bug template.

Hybrid intent (13.33%): Issues that could have multiple defensible labels due to their combined intents. These errors mostly take place when a fix or a feature request is layered into a question or bug report. For example, in

⁷<https://github.com/microsoft/vscode/issues/193985>

kubernetes#120364⁸ the author both reports a concrete bug and in the same body explicitly requests a new feature: “Please, give us multiarchitecture image! and fix monoarchitecture images too!”

Label noise (9.33%): Issues whose ground-truth label does not match their content. For example, tensorflow#61710⁹ is titled “Will there be an MLP model in the future version?” which requests a new API feature for building MLPs. Yet the dataset labels it as a bug.

We also sampled 30 *invalid* predictions each from RAG and filtered RAG (60 total) and observed three failure modes. In roughly 73% of cases, the model either continued to generate issue-body content or its own reasoning. Another 23% were hallucinations such as repeated digits and punctuation. The remaining 3% are parsable labels that fall outside the three valid classes (e.g., documentation).

B. Key Insights and Practical Implications

The practicality of context retrieval for IRC: Our results indicate that both RAG variants offer a practical solution for IRC. With 11× less labeled data per project and 33% less peak GPU memory on average, RAG trails fine-tuning by 2.8 points in aggregate macro F_1 , and the 1.0-point gap to filtered RAG is not statistically significant. With a k NN voting fallback for invalid outputs, filtered RAG matches fine-tuning (Section IV-D). The RAG variants offer deployment advantages over fine-tuning as well: adding a new labeled issue report becomes an incremental index update rather than a retraining cycle, and the model can be swapped in production with no project-specific retraining. For extremely resource-constrained scenarios, even k NN voting with the target project’s issues is a defensible baseline: it reaches 59.5% macro F_1 ($k=15$), within 1.8 points of Qwen-3B’s zero-shot 61.3% (Section IV-A, Section IV-B), at a fraction of the GPU cost (≈ 240 MB vs 2.9 GB).

Combining classification and validation: Our failure analysis (Section V-A) shows that users frequently file support questions and feature requests under the bug-report template, which can bias the classifier towards predicting bug regardless of the content. This emphasizes the importance of refining the new LLM-assisted triage pipelines now in place on platforms such as GitHub [32] and GitLab [33]. In these pipelines, the reporter describes the report to an LLM, which then suggests the correct template and labels. Report validation is also important since mislabeling and duplication are common problems among contributors [8], [15]. Studies have already explored validation for bug reports [34]–[37]. However, combining classification and validation in a cohesive workflow is underexplored.

VI. RELATED WORK

Automated Issue Report Classification (IRC): Early IRC studies employed data mining and ML classifiers [7], [8], [10]. However, these methods yielded limited performance due to

their shallow semantic understanding of issue report text [14]. To overcome this limitation, more recent work has adopted transformer models for their stronger contextual understanding [11], [16], [17], [38], [39]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [11] and Trautsch et al. [16] fine-tuned RoBERTa [40] and seBERT [41] models, respectively, to classify issues into bugs, features, or questions. Although transformer-based approaches achieve strong results, they rely on large amounts of training data, which limits their generalizability to settings where labeled data is scarce [11], [14], [17], [39]. These challenges have motivated the exploration of LLMs for IRC. Their extensive pretraining allows these models to be applied with little or no task-specific training, reducing the dependence on the large labeled datasets that supervised classifiers require [1], [6], [13], [19], [42]. However, state-of-the-art LLM-based methods do not leverage this property. Instead, they fine-tune the LLM with Low-Rank Adaptation (LoRA [20]) on labeled issue datasets to achieve their best reported performance [6], [14], [19]. Although fine-tuning is effective, it reintroduces the dependency on labeled training data, remains computationally expensive, and must be repeated whenever the underlying model is swapped or as newly labeled issues accumulate in the project [7], [10].

Retrieval-Augmented Few-shot Prompting: Studies show that few-shot prompting can improve LLM performance in specialized tasks due to in-context learning [21], [43]–[46]. Few-shot prompting has also been applied in IRC. Colavito et al. [1], [13] randomly selected one or two labeled examples per class from their training data and presented them in a few-shot prompt. In their benchmark, these examples did not improve over zero-shot prompting for most models [13]. Few-shot performance is sensitive to the quality of in-context examples. Research shows that using retrieval techniques to select the most semantically relevant in-context examples can outperform random selection [22], [23]. Since retrieval-augmented few-shot prompting can enhance performance without the overhead of training, it can be a promising alternative to the LoRA fine-tuning IRC baseline. Recent IRC studies also point in this direction in their future work. Heo and Lee [14] suggest few-shot prompting to improve accuracy across diverse projects, and Aracena et al. [6] propose retrieval augmentation, in which relevant past issues are retrieved and added to the LLM’s prompt, to reduce the need for extensive fine-tuning. To the best of our knowledge, however, this method has not been systematically and empirically evaluated for IRC.

Question-to-Bug Misclassification: In IRC studies that include both bug and question labels, question is often the hardest class to identify, and misclassified questions are most often labeled as bugs. For example, almost all methods in the NLBSE’24 tool competition struggled with question [12], and on the same dataset, the two best-performing zero-shot LLMs labeled 46–54% of questions as bugs [13]. Fine-tuned decoder LLMs also score lowest on question in aggregate [6], [14]. Our results show the same pattern. Every method we evaluate

⁸<https://github.com/kubernetes/kubernetes/issues/120364>

⁹<https://github.com/tensorflow/tensorflow/issues/61710>

misclassifies questions as bugs, with k NN voting predicting bug for 32% of questions, zero-shot prompting for 46–59%, and PA fine-tuning for 23–36% (Sections IV-A, IV-B and IV-D). As a remedy, Aracena et al. [6] suggested retrieval augmentation, particularly for ambiguous labels such as question. Our RAG variants show that it helps but does not completely eliminate the problem. Without any training, retrieved examples lower the share of questions predicted as bugs from 46–59% to 26–36%, and filtered RAG lowers it further to 19–27%.

Prior work offers several explanations for why questions are misclassified as bugs. A common reason is label imbalance, since question is often the smallest class in the training data [11], [38]. Another explanation is that some questions read like bug reports because they describe unexpected behavior, contain error messages [25], or lack explicit interrogative markers [6]. Our failure analysis (Section V-A) shows the same effect for questions filed under a bug-report template, which led the model to predict bug regardless of their content. Finally, a label can depend on information outside the report, such as a maintainer’s later diagnosis [25]. Likewise, the label-noise cases in our failure analysis have labels that do not match their content and cannot be inferred from the report text alone.

VII. THREATS TO VALIDITY

Internal Validity: Filtered RAG’s margin uses the same empirically derived fixed value across all models and projects with no per-project or per-model tuning, and is not framed as a universally optimal constant. Furthermore, the choice of hyperparameters and prompt format can also pose an internal threat to validity as different hyperparameters and prompts can change fine-tuning performance [6], [14].

External Validity: Issue report text can vary substantially in writing style, vocabulary, and structure across software projects [47], which may threaten the generalizability of our findings. Another threat to external validity is generalizability across settings. We note that many configurations, including LLM architecture, prompt structure, embedding model, and retrieval technique, can each lead to different results. Assessing how far our findings generalize to other settings is itself an important question, but one that falls outside this paper’s scope and warrants its own dedicated studies.

Construct Validity: The inherent randomness of LLM outputs can pose a potential threat to construct validity. The reported performance metrics can vary between different runs due to LLM randomness. To mitigate this variance, we ran the experiments three times and report the average.

VIII. CONCLUSION AND FUTURE WORK

In this study, we investigated retrieval-augmented few-shot prompting (RAG) as a training-free alternative to LoRA fine-tuning for issue report classification (IRC). We conducted an empirical study with four Qwen2.5-Instruct models and 6,600 issue reports from eleven projects, where the retrieval index and the fine-tuning data come either from the target project’s 300 labeled issues or from all 3,300 labeled issues of the eleven

projects. We first tested how well the labels of a query issue’s most similar neighbors predict its label, using a similarity-weighted vote over them (k NN voting). Since these neighbors carry label information, RAG gives them to the LLM as few-shot examples. Because RAG most often labels questions as bugs, a filtered variant (filtered RAG) removes the bug-labeled examples for suspected questions.

Retrieved examples improve the LLM over both zero-shot prompting and k NN voting, and filtered RAG improves on RAG by reducing question-to-bug errors, at the cost of bug recall. With the same labeled issues of the target project, RAG and filtered RAG at their best number of examples outperform LoRA fine-tuning at every model size. When fine-tuning instead uses all 3,300 labeled issues of the eleven projects, it outperforms RAG, while its lead over filtered RAG is not statistically significant. With k NN voting replacing invalid LLM outputs, filtered RAG matches fine-tuning (0.1 point ahead in macro F_1 on average). Fine-tuning finds more of the actual bugs but over-predicts the bug label, whereas filtered RAG makes fewer false bug predictions at most model sizes, and the question-to-bug confusion persists under every method we test. Needing only the target project’s labeled issues and 33% less peak GPU memory on average, retrieval-augmented few-shot prompting is a competitive, training-free alternative to LoRA fine-tuning.

The two approaches trade different strengths, and which one suits a project depends on its labeled data, its GPU budget, and whether a missed bug or a false bug label costs it more. With retrieval, a newly labeled issue only needs to be indexed, and the LLM can be replaced without retraining. As this work constitutes a case study on the Qwen2.5-Instruct family, future work can focus on the generalizability of these findings by extending them to other LLM families and quantization settings to further strengthen or bound the conclusions drawn here. Additional directions include examining imbalanced label distributions, minority-class scenarios, multi-label classification, and other issue tracking platforms such as Jira and Bugzilla. Further research should also explore stronger retrieval-side interventions, such as LLM-based filtering or re-ranking of retrieved neighbors, as well as the effects of different embedding models and similarity metrics on retrieval quality. Finally, future studies should investigate alternative prompt structures, including advanced strategies such as Chain-of-Thought prompting [48].

DATA AVAILABILITY

Our replication package [49] contains the code, data, and all necessary materials to reproduce the results presented in this paper.

REFERENCES

- [1] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Leveraging gpt-like llms to automate issue labeling," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 469–480.
- [2] Z. Liao, D. He, Z. Chen, X. Fan, Y. Zhang, and S. Liu, "Exploring the characteristics of issue-related behaviors in github using visualization techniques," *IEEE Access*, vol. 6, pp. 24003–24015, 2018.
- [3] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, "Not all bugs are the same: Understanding, characterizing, and classifying bug types," *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019.
- [4] H. Wu, Y. Ma, Z. Xiang, C. Yang, and K. He, "A spatial-temporal graph neural network framework for automated software bug triaging," *Knowledge-Based Systems*, vol. 241, p. 108308, 2022.
- [5] F. Santos, J. Vargovich, B. Trinkenreich, I. Santos, J. Penney, R. Britto, J. F. Pimentel, I. Wiese, I. Steinmacher, A. Sarma *et al.*, "Tag that issue: applying api-domain labels in issue tracking systems," *Empirical Software Engineering*, vol. 28, no. 5, p. 116, 2023.
- [6] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models to issue classification: Revisiting with extended data and new models," *Science of Computer Programming*, vol. 246, p. 103333, 2025.
- [7] R. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, "Ticket tagger: Machine learning driven issue classification," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2019, pp. 406–409.
- [8] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, and Y.-G. Guéhéneuc, "Is it a bug or an enhancement? a text-based approach to classify change requests," in *Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds*, 2008, pp. 304–318.
- [9] K. Herzig, S. Just, and A. Zeller, "It's not a bug, it's a feature: how misclassification impacts bug prediction," in *2013 35th international conference on software engineering (ICSE)*. IEEE, 2013, pp. 392–401.
- [10] Q. Fan, Y. Yu, G. Yin, T. Wang, and H. Wang, "Where is the road for issue reports classification based on text mining?" in *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 2017, pp. 121–130.
- [11] M. Izadi, "CatIss: An intelligent tool for categorizing issues reports using transformers," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 44–47.
- [12] R. Kallis, G. Colavito, A. Al-Kaswan, L. Pascarella, O. Chaparro, and P. Rani, "The NLBSE'24 tool competition," in *Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering*, 2024, pp. 33–40.
- [13] G. Colavito, F. Lanubile, and N. Novielli, "Benchmarking large language models for automated labeling: The case of issue report classification," *Information and Software Technology*, vol. 184, p. 107758, 2025.
- [14] J. Heo and S. Lee, "A study on applying large language models to issue classification," in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*. IEEE Computer Society, 2025, pp. 1–11.
- [15] N. Pandey, D. K. Sanyal, A. Hudait, and A. Sen, "Automated classification of software issue reports using machine learning techniques: an empirical study," *Innovations in Systems and Software Engineering*, vol. 13, no. 4, pp. 279–297, 2017.
- [16] A. Trautsch and S. Herbold, "Predicting issue types with sebert," in *Proceedings of the 1st international workshop on natural language-based software engineering*, 2022, pp. 37–39.
- [17] G. Colavito, F. Lanubile, and N. Novielli, "Issue report classification using pre-trained language models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 29–32.
- [18] J. Wang, X. Zhang, and L. Chen, "How well do pre-trained contextual language representations recommend labels for github issues?" *Knowledge-Based Systems*, vol. 232, p. 107476, 2021.
- [19] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models to issue classification," in *Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering*, 2024, pp. 57–60.
- [20] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models," *Iclr*, vol. 1, no. 2, p. 3, 2022.
- [21] T. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [22] J. Liu, D. Shen, Y. Zhang, W. B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for gpt-3?" in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*, 2022, pp. 100–114.
- [23] G. Yu, L. Liu, H. Jiang, S. Shi, and X. Ao, "Retrieval-augmented few-shot text classification," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 6721–6735.
- [24] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
- [25] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Impact of data quality for automatic issue classification using pre-trained language models," *Journal of Systems and Software*, vol. 210, p. 111838, 2024.
- [26] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.
- [27] H. Huang, R. Widyasari, T. Zhang, I. C. Irsan, B. Shi, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang *et al.*, "Back to the basics: Rethinking issue-commit linking with llm-assisted retrieval," *arXiv preprint arXiv:2507.09199*, 2025.
- [28] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The FAISS library," *IEEE Transactions on Big Data*, 2025.
- [29] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE transactions on information theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [30] S. A. Dudani, "The distance-weighted k-nearest-neighbor rule," *IEEE Transactions on Systems, Man, and Cybernetics*, no. 4, pp. 325–327, 1976.
- [31] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.
- [32] GitHub, "GitHub Copilot," <https://github.com/features/copilot>, 2025.
- [33] GitLab, "GitLab Duo," <https://about.gitlab.com/gitlab-duo/>, 2025.
- [34] L. Khaib, N. S. Mathews, and M. Nagappan, "Assertflip: Reproducing bugs via inversion of llm-generated passing tests," *arXiv preprint arXiv:2507.17542*, 2025.
- [35] X. Wang, P. Gao, X. Meng, C. Peng, R. Hu, Y. Lin, and C. Gao, "Aegis: An agent-based framework for general bug reproduction from issue descriptions," *arXiv preprint arXiv:2411.18015*, 2024.
- [36] T. Ahmed, J. Ganhotra, R. Pan, A. Shinnar, S. Sinha, and M. Hirzel, "Otter: Generating tests from issues to validate swe patches," *arXiv preprint arXiv:2502.05368*, 2025.
- [37] E. Dinç and E. Tüzün, "Judge the votes: A system to classify bug reports and give suggestions," in *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 2025, pp. 01–10.
- [38] S. Bharadwaj and T. Kadam, "GitHub issue classification using BERT-style models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 40–43.
- [39] M. Izadi, K. Akbari, and A. Heydarnoori, "Predicting the objective and priority of issue reports in software repositories," *Empirical Software Engineering*, vol. 27, no. 2, p. 50, 2022.
- [40] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [41] J. Von der Mosel, A. Trautsch, and S. Herbold, "On the validity of pre-trained transformers for natural language processing in the software engineering domain," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1487–1507, 2022.
- [42] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Large language models for issue report classification," in *Ital-IA*, 2024, pp. 18–23.
- [43] M. Assi, S. Hassan, and Y. Zou, "Llm-cure: Llm-based competitor user review analysis for feature enhancement," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 4, pp. 1–25, 2026.
- [44] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2438–2449.
- [45] H. Ma, C. Zhang, Y. Bian, L. Liu, Z. Zhang, P. Zhao, S. Zhang, H. Fu, Q. Hu, and B. Wu, "Fairness-guided prompt-based few-shot prompting for large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2303.13217>

- [46] R. L. L. IV, I. Balažević, E. Wallace, F. Petroni, S. Singh, and S. Riedel, "Cutting down on prompts and parameters: Simple few-shot learning with language models," 2021. [Online]. Available: <https://arxiv.org/abs/2106.13353>
- [47] A. Akhavan, A. Hoseinpour, A. Heydarnoori, H. Bagheri, and M. Keshani, "LinkAnchor: An autonomous LLM-based agent for issue-to-commit link recovery," in *To Appear in Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, July 2026, preprint available at <https://arxiv.org/abs/2508.12232>.
- [48] A. Koyuncu, "Exploring fine-grained bug report categorization with large language models and prompt engineering: An empirical study," *ACM Transactions on Software Engineering and Methodology*, 2025.
- [49] "Replication package: Can retrieval-augmented few-shot prompting match lora fine-tuning for issue report classification? an empirical study," <https://figshare.com/s/2c7a6db860c611246774>, Accessed: 05-18-2026.